Counting Items in Amazon Fulfilment-Centre Bins from a Photo

UE24CS352A Machine Learning · Mini-Project · Problem #65 · [Name 1 (SRN)], [Name 2 (SRN)]

Code: [GitHub repository URL]

1. Problem statement

Items in Amazon bins are sometimes misplaced, so the recorded inventory differs from the real contents. Given one colour photo of a bin, we predict the number of items it holds (0 to 5, a 6-class problem; random-guess baseline ≈ 16.7%). If the prediction disagrees with the recorded count, the bin is flagged for a recount, which supports inventory reconciliation.

2. Dataset

Amazon Bin Image Dataset (AWS Open Data): 535,234 photos of bins in operating fulfilment centres, each with JSON metadata whose *EXPECTED_QUANTITY* field is our label. We use a random subset of **[N]** images, stratified 70/20/10 into train/val/test (*fill in by running the pipeline and re-building this PDF*). We keep bins with at most 5 items, resize to 224×224 and normalise. Images are noisy: tape and plastic wrap occlude items, items overlap, and lighting varies, so some bins are hard even for humans.

3. Approach

Classical baseline. Downsampled pixels → standardisation → PCA → Logistic Regression and an RBF-kernel SVM; C and γ are tuned on the validation split. **Deep model.** ImageNet-pretrained ResNets (18/34) with a new 6-way softmax head, trained with cross-entropy. Phase 1 trains only the head; phase 2 fine-tunes the whole network with differential learning rates (lr/9 early layers, lr/3 later layers, lr for the head) and cosine annealing, comparing SGD and Adam. Augmentation: horizontal flips and mild colour jitter; flip test-time augmentation (TTA) is reported separately. The model with the best validation accuracy is evaluated once on the held-out test split.

4. Implementation overview

Python, scikit-learn and PyTorch/torchvision. `download_data.py` fetches and filters the subset and builds the splits; `baseline_svm.py` and `train_cnn.py` train and evaluate, writing metrics (accuracy, RMSE of the predicted count, per-class accuracy, confusion matrix) to `results/`; `demo.py` loads a checkpoint, predicts the count for a photo and flags any mismatch with the recorded count. A smoke-test suite runs the pipeline on synthetic data. This PDF is generated from the result files, so every number below comes from the code in the repository.

5. Results (held-out test split)

Model	Val acc	Test acc	Test RMSE	Test acc (flip-TTA)
Logistic Regression + PCA	TBD	TBD	TBD	TBD
SVM (RBF) + PCA	TBD	TBD	TBD	TBD
ResNet-18	TBD	TBD	TBD	TBD
ResNet-34	TBD	TBD	TBD	TBD

Results pending: run the training scripts, then re-run `python report/build_report.py` to fill this table.

Reference point from the literature (Bertorello et al., Stanford): on a similar 0–5 task, SVMs reached about 26–32% and ResNet-34 about 56% accuracy. These are context only, not our results.

6. Conclusions

To be written once results exist. Expected discussion: CNN vs classical models, effect of optimiser/fine-tuning, per-class behaviour and its causes, limitations (occlusion, label noise), and future work.